

MANUAL DEL ASISTENTE

Skills para Claude Code

Políticas, reglas, buenas prácticas y cómo se crean

Webinar · 30 de septiembre de 2026
Erick Vázquez Granados

Contenido

Cómo leer este manual	3
PARTE I • Fundamentos	4
1. Qué es una skill	4
2. El problema que resuelve	4
3. El detalle que casi nadie enseña	5
4. Dónde vive una skill	5
5. Skills y comandos son lo mismo	5
PARTE II • Cómo se crean	6
6. El procedimiento, paso a paso	6
7. La descripción • el campo que decide todo	7
8. Campos del frontmatter	8
9. Grados de libertad	9
10. Partir una skill grande	9
11. Contexto dinámico	10
PARTE III • Reglas y políticas	11
12. Políticas de creación	11
13. Políticas de seguridad	12
14. Política de revisión en equipo	12
15. Política de ciclo de vida	13
PARTE IV • Buenas prácticas	14
16. Concisión	14
17. Da una opción, no cinco	14
18. Escribe para el modelo más chico	14
19. Evalúa antes de documentar	14
20. Prueba siempre en sesión nueva	14
21. Diagnóstico rápido	15
22. Antipatrones que no dan error	15
23. Qué capa usar	16
Documentación oficial	16

Políticas, reglas, buenas prácticas y cómo se crean

OmniSys · Dirección de Innovación y Preventa

Cómo leer este manual

Cuatro partes, en orden de urgencia:

PARTE	QUÉ RESUELVE	CUÁNDO LA LEES
I. Fundamentos	Qué es una skill y por qué funciona	Hoy
II. Cómo se crean	El procedimiento, paso a paso	Hoy, con la terminal abierta
III. Reglas y políticas	Qué se permite, qué no, y cómo se revisa en equipo	Antes de compartir tu primera skill
IV. Buenas prácticas	Lo que separa una skill que se usa de una que se abandona	Cuando ya tengas dos o tres

Si solo tienes quince minutos, lee la parte II y construye una. Todo lo demás cobra sentido después de eso.

PARTE I · Fundamentos

1. Qué es una skill

Un directorio con un archivo `SKILL.md` adentro:

```
mi-skill/  
└─ SKILL.md
```

El archivo tiene dos partes: metadatos en YAML entre marcadores `---`, y debajo instrucciones en markdown.

```
---  
description: Resume los cambios sin commitear y señala lo riesgoso.  
            Úsala cuando el usuario pregunte qué cambió, pida un mensaje de  
            commit o quiera revisar su diff.  
---  
  
## Cambios actuales  
  
!`git diff HEAD`  
  
## Instrucciones  
  
Resume en dos o tres viñetas qué cambió. Luego lista los riesgos:  
manejo de errores ausente, valores hardcodeados, pruebas  
desactualizadas.
```

No hay registro, no hay instalación, no hay build. Creas la carpeta, escribes el archivo, y Claude Code lo detecta en la sesión en curso sin reiniciar.

2. El problema que resuelve

Claude tiene una ventana de contexto finita. Todo lo que quieres que sepa compite por ese espacio: tu código, tu conversación, tus convenciones.

Las skills parten el contenido en dos niveles:

- **La descripción** vive permanentemente en el contexto. Es corta. Le dice a Claude *que la skill existe y cuándo aplicarla*.
- **El cuerpo** solo entra cuando la skill se usa. Puede tener cientos de líneas y no cuesta nada mientras no se invoque.

Podrías tener cincuenta skills instaladas. Lo permanente son cincuenta descripciones cortas. Los cincuenta cuerpos no están ahí hasta que hacen falta.

Esta es la idea central. Si la entiendes, el resto se deduce. Si no, vas a escribir skills que no se activan y no vas a saber por qué.

3. El detalle que casi nadie enseña

Cuando una skill se invoca, **su contenido entra a la conversación y se queda ahí en los turnos siguientes**. No es un préstamo momentáneo. Claude Code no vuelve a leer el archivo después.

Tres consecuencias prácticas:

- 1. Cada línea del cuerpo es un **costo recurrente** de tokens, no un costo único.
- 2. Escribe **instrucciones permanentes**, no pasos de un solo uso. Si algo debe aplicar durante toda la tarea, redáctalo como norma vigente.
- 3. Si vuelves a invocar la misma skill y el contenido es idéntico, no se duplica. Si cambió —por argumentos distintos o contexto dinámico nuevo— sí se agrega otra vez.

Cuando el contexto se llena y hay autocompactación, Claude Code re-adjunta la invocación más reciente de cada skill conservando los primeros 5,000 tokens de cada una, con un presupuesto combinado de 25,000. Llena ese presupuesto desde la más reciente, así que **las skills invocadas al principio de una sesión larga pueden desaparecer**.

Si sientes que una skill "dejó de funcionar" a media conversación, casi siempre el contenido sigue ahí y el modelo está eligiendo otro camino. Refuerza la descripción, o vuelve a invocarla.

4. Dónde vive una skill

UBICACIÓN	RUTA	APLICA A
Empresarial	Configuración administrada	Toda la organización
Personal	<code>~/.claude/skills/<nombre>/SKILL.md</code>	Todos tus proyectos
Proyecto	<code>.claude/skills/<nombre>/SKILL.md</code>	Ese repositorio
Plugin	<code><plugin>/skills/<nombre>/SKILL.md</code>	Donde el plugin esté activo

El nivel de proyecto es el que cambia la vida en equipo. Se versiona con el código: quien clona el repositorio hereda tu criterio sin que se lo expliques.

En conflicto de nombres, empresarial gana sobre personal, y personal gana sobre proyecto. Las skills de plugin usan espacio de nombres propio (`plugin:skill`) y no chocan con nadie.

El comando que escribes sale del nombre del directorio, no del campo `name`. En skills personales y de proyecto, `name` es solo la etiqueta del listado, y ni siquiera es obligatorio.

5. Skills y comandos son lo mismo

Históricamente había dos sistemas: comandos en `.claude/commands/*.md` y skills en `.claude/skills/*/SKILL.md`. **Se fusionaron.** Ambos crean `/nombre` y funcionan igual.

Las skills agregan tres cosas: un directorio para archivos de apoyo, frontmatter para controlar quién las invoca, y carga automática por el modelo. Un comando personalizado es hoy una skill sin funciones extra.

PARTE II · Cómo se crean

6. El procedimiento, paso a paso

Paso 1 · Elige un caso real

No inventes uno bonito. Responde esta pregunta:

¿Qué le pido a Claude cada semana, escribiendo más o menos lo mismo?

Si no se te ocurre nada, revisa tu historial de sesiones, los últimos PR que revisaste, o lo último que le explicaste a alguien de tu equipo.

Paso 2 · Resuélvelo a mano primero

Abre una sesión y resuelve la tarea con prompting normal, sin skill:

```
claude
```

Fíjate en qué contexto tuviste que dar: qué criterio, qué formato, qué te importaba. Eso es exactamente lo que va a la skill.

Paso 3 · Pídele a Claude que la escriba

Cuando tengas un resultado que te sirva:

```
crea una skill que capture este patrón
```

Nadie escribe un `SKILL.md` bueno de memoria y a la primera. Que lo genere él y tú lo recortas.

Paso 4 · Recorta

Claude va a generar de más. Quítale todo lo que ya sabe:

```
quita la explicación de qué es un pull request, eso ya lo sabes
```

```
déjalo en menos de 30 líneas
```

Paso 5 · Crea la carpeta

```
mkdir -p ~/.claude/skills/resumiendo-cambios
```

El nombre del directorio es el comando. Gerundio que describe la actividad.

Paso 6 · Pruébala en limpio

Sal y abre una sesión **nueva**. Pide la tarea en lenguaje natural, **sin escribir** / :

¿qué cambié hoy?

¿Se disparó sola?

Paso 7 · La vuelta

- **No se disparó** → el problema está en la descripción, casi nunca en el cuerpo
- **Se disparó pero hizo otra cosa** → el problema sí está en el cuerpo

Vuelves a la sesión original, corriges, y pruebas otra vez en una nueva.

A este ciclo lo llamamos **Claude A / Claude B**. A es donde construyes, B es donde pruebas en limpio. Lo que observas en B regresa a A. Es el trabajo real de escribir skills.

7. La descripción · el campo que decide todo

Es lo único que Claude ve para decidir si tu skill aplica. Si está mal escrita, la skill **no existe** en la práctica.

La fórmula

[qué hace] + [cuándo usarla]

Ambas mitades son obligatorias.

Las cuatro reglas

1 · Tercera persona. La descripción se inyecta en la indicación del sistema. Escribirla como si Claude le hablara al usuario rompe el descubrimiento.

```
# ✗ nunca se activa
description: Te puedo ayudar con tus documentos y archivos.

# ✔ se dispara sola
description: Analiza documentos y aplica el formato del equipo.
Úsala cuando el usuario mencione README, documentación o formato.
```

2 · El caso principal al inicio. El listado de skills tiene tope de caracteres. Si tu texto se recorta, se recorta **por el final**.

3 · Las palabras que la gente escribe de verdad. No las del manual. Nadie teclea "gestión documental del repositorio"; te clean "revisa este README".

Truco: escribe primero cómo pedirías la tarea, en voz alta, y de ahí saca las palabras.

4 · El tope es de 1,536 caracteres, combinado entre `description` y `when_to_use`.

Patologías y su síntoma

DESCRIPCIÓN ENFERMA	SÍNTOMA
Te puedo ayudar con X	Nunca se activa sola
Ayuda con documentos	Se activa cuando no debe, o nunca
Sin la parte del "cuándo"	Claude sabe qué hace pero no cuándo
Larga, con el caso principal al final	Se trunca y pierde lo que importa
Con jerga que nadie teclea	Nunca coincide con la petición real
Solapada con otra skill	Se dispara la equivocada, inconsistente

8. Campos del frontmatter

Todos son opcionales. Solo `description` es recomendado. Claude Code lee el frontmatter **únicamente si el `---` de apertura es la primera línea.**

CAMPO	PARA QUÉ	CUÁNDO LO USAS
<code>description</code>	Qué hace y cuándo usarla	Siempre
<code>when_to_use</code>	Frases disparadoras adicionales	Cuando la descripción se alarga
<code>name</code>	Etiqueta de listado	Opcional
<code>allowed-tools</code>	Preaprueba herramientas durante el turno	Cuando la skill ejecuta comandos
<code>disallowed-tools</code>	Retira herramientas mientras está activa	Skills autónomas
<code>disable-model-invocation</code>	Solo tú la invocas	Acciones con efectos
<code>user-invocable</code>	<code>false</code> : solo Claude la invoca	Conocimiento de fondo
<code>argument-hint</code>	Pista de autocompletado	Skills con argumentos
<code>model</code>	Modelo mientras está activa	Tareas que necesitan otro modelo
<code>context: fork</code>	Corre en subagente aislado	Tareas que ensucian el contexto
<code>paths</code>	Globs que limitan la activación	Monorepos

`allowed-tools` no restringe

Esta es la confusión número uno.

`allowed-tools` **concede** permiso para que Claude no te pregunte. **No restringe**: todas las herramientas siguen disponibles. El permiso se limpia cuando envías tu siguiente mensaje.

Para restringir de verdad: `disallowed-tools` mientras la skill está activa, o reglas de negación en permisos para siempre.

Los dos campos de invocación

FRONTMATTER	TÚ INVOCAS	CLAUDE INVOCA	CONTEXTO
(por defecto)	Sí	Sí	Descripción siempre; cuerpo al invocar
<code>disable-model-invocation: true</code>	Sí	No	Descripción NO está en contexto
<code>user-invocable: false</code>	No	Sí	Descripción siempre; cuerpo al invocar

9. Grados de libertad

La pregunta no es "¿cuánto detalle pongo?" sino "¿qué tan frágil es esta tarea?".

GRADO	SITUACIÓN	CÓMO SE ESCRIBE	EJEMPLO
Alta	Varios caminos sirven	Prosa, dirección general	Revisión de código
Media	Hay un patrón preferido	Plantilla o script con parámetros	Generar un reporte
Baja	Se puede romper algo	Comando exacto y prohibición de variarlo	Migración, despliegue

Ejemplo de baja libertad escrito correctamente:

```
Ejecuta exactamente:  
  
python scripts/migrate.py --verify --backup  
  
No modifiques el comando ni agregues banderas, aunque el mensaje de  
error lo sugiera. Si falla, detente y reporta. No intentes corregirlo.
```

La última línea es parte del diseño, no un adorno. Sin ella, un modelo capaz va a intentar arreglar el problema, que es lo que no quieres en una migración.

El error típico: escribir todas tus skills con el mismo nivel de detalle. Las de alta libertad quedan asfixiadas y las de baja quedan peligrosamente vagas.

10. Partir una skill grande

```
mi-skill/  
├─ SKILL.md           ← el procedimiento  
├─ reference.md       ← el detalle, solo si hace falta  
└─ scripts/  
    └─ revisar.sh     ← se EJECUTA, no se lee
```

Tres reglas donde la gente se equivoca:

1. Un solo nivel. `SKILL.md` → `reference.md`. Nunca una cadena de tres. Las cadenas funcionan a veces y fallan otras, que es peor que fallar siempre.

2. Los scripts se ejecutan. Si le pides a Claude que lea el script y lo interprete, gastas contexto en algo que la máquina hacía sola.

3. Rutas con `${CLAUDE_SKILL_DIR}`. Nunca absolutas, nunca con barra invertida.

El patrón que evita las solicitudes de permiso

Claude Code sustituye `${CLAUDE_SKILL_DIR}` en **dos lugares**: el cuerpo markdown y las reglas de Bash en `allowed-tools`. Usar la misma variable en ambos hace que la regla coincida exactamente con el comando:

```
---
description: Renderiza una gráfica a partir de un CSV
allowed-tools: Bash(`${CLAUDE_SKILL_DIR}/scripts/render.sh *)
---

Ejecuta `${CLAUDE_SKILL_DIR}/scripts/render.sh <archivo-csv>`.
```

El script corre **sin preguntarte permiso**. Sin este patrón, cada ejecución interrumpe.

Límite práctico: `SKILL.md` por debajo de 500 líneas.

11. Contexto dinámico

`! comando` ejecuta el comando **antes** de que el contenido llegue a Claude y reemplaza el marcador con la salida:

```
## Cambios actuales
!`git diff HEAD`

## Estado
!`git status --short`
```

La diferencia con pedirle que corra `git diff` es sustancial: no gasta un turno de herramienta, no puede decidir no hacerlo, y la instrucción llega con los datos adentro.

La regla que te va a morder

Un comando que falla aborta toda la invocación. Claude nunca ve el contenido de la skill.

Con `bash`, cualquier código de salida distinto de cero cuenta como fallo. Si tu comando puede salir con error por razones normales, anéxale `|| true`:

```
## Estado del linter
!`npm run lint || true`
```

Además, los comandos inyectados **nunca piden permiso**: si la verificación de permisos no devuelve "permitir", se aborta. Pre-apruébalo con `allowed-tools`.

PARTE III · Reglas y políticas

Esta parte es para cuando las skills dejan de ser tuyas y pasan a ser del equipo. Adáptala; está escrita para copiarse a la documentación interna.

12. Políticas de creación

P-01 · Nomenclatura obligatoria

- El directorio se nombra en **gerundio que describe la actividad**: `revisando-prs`, `generando-migraciones`
- Solo minúsculas, números y guiones. Máximo 64 caracteres
- No puede contener las palabras `anthropic` ni `claude`
- Prohibidos los nombres genéricos: `helper`, `utils`, `tools`, `misc`, `general`

P-02 · Toda skill declara su grado de libertad

Como comentario al final del `SKILL.md`:

```
<!-- Grado de libertad: ALTA / MEDIA / BAJA -->
```

Es lo que permite revisar si el nivel de detalle es el correcto.

P-03 · Acciones con efectos llevan `disable-model-invocation`

Cualquier skill que despliegue, committee, envíe mensajes, borre datos o modifique estado externo **debe** llevarlo. No se deja que el modelo decida ejecutar algo irreversible.

P-04 · Sin información con fecha de caducidad

Prohibido escribir versiones concretas, precios, nombres de personas en puestos, o cualquier dato que vaya a envejecer. Si hace falta, se inyecta con contexto dinámico.

P-05 · Sin secretos

Ninguna skill contiene tokens, credenciales, cadenas de conexión, rutas internas de infraestructura ni datos de clientes. Las skills se versionan: lo que escribas ahí queda en el historial.

P-06 · Terminología consistente

Un concepto, un término, de principio a fin del archivo. Si arriba dice "documento" y abajo "archivo", Claude no sabe si son lo mismo.

13. Políticas de seguridad

S-01 · Revisión obligatoria de `allowed-tools` en skills ajenas

La confianza del espacio de trabajo **no gobierna** `allowed-tools`. Una skill de proyecto aplica su preaprobación aunque la carpeta nunca se haya marcado como confiable.

Antes de correr Claude Code en un repositorio que no es tuyo, revisa el `allowed-tools` de sus skills. Si no entiendes por qué una skill necesita lo que pide, no la corras.

S-02 · Principio de mínima preaprobación

`allowed-tools` lista el comando más específico posible:

```
# ❌ demasiado amplio
allowed-tools: Bash

# ✅ acotado
allowed-tools: Bash(git status *) Bash(git diff *)
```

S-03 · Los comandos inyectados son código que corre en tu máquina

Todo `! comando` en una skill se ejecuta sin preguntar. Trátales con el mismo criterio con que aceptarías un script en un PR.

S-04 · Skills que tocan sistemas productivos

Requieren: `disable-model-invocation: true`, verificación previa con script, grado de libertad BAJA, y revisión por una segunda persona antes de mezclarse.

14. Política de revisión en equipo

Una skill nueva en `.claude/skills/` se revisa como se revisa código. Checklist mínimo del revisor:

- ¿La descripción está en tercera persona?
- ¿Dice qué hace y cuándo usarla?
- ¿Usa las palabras que el equipo escribe de verdad?
- ¿El nombre del directorio describe la actividad?
- ¿Declara su grado de libertad y es el correcto?
- ¿`SKILL.md` por debajo de 500 líneas?
- ¿Las referencias van a un solo nivel?
- ¿`allowed-tools` pide lo mínimo?
- Si tiene efectos, ¿lleva `disable-model-invocation`?
- ¿Sin secretos, sin datos de cliente, sin información con caducidad?
- ¿El revisor la ejecutó sin que el autor le explicara nada?

El último punto es el que más vale. Si necesitas explicación, la descripción no fue suficiente.

15. Política de ciclo de vida

MOMENTO	QUÉ PASA
Nace	Personal, en <code>~/ .claude/skills/</code> . Nadie más la ve
Se prueba	Una semana de uso real del autor. Mínimo tres invocaciones
Se propone	PR a <code>.claude/skills/</code> del proyecto, con el checklist de arriba
Vive	Se revisa cada vez que cambia el proceso que describe
Muere	Si el proceso desapareció, se borra. Una skill que miente es peor que ninguna

Señal de que hay que jubilar una skill: nadie la ha invocado en dos meses y el equipo resolvió el problema de otra forma.

PARTE IV · Buenas prácticas

16. Concisión

La ventana de contexto es un bien común, y el cuerpo de la skill **permanece turno tras turno**. Ese costo es recurrente.

Regla: solo agrega lo que Claude no sabe ya.

- Si tu skill explica qué es un pull request → bórralo
- Si explica las convenciones de nombres de **tu** equipo → déjalo
- Enuncia **qué hacer**, no narres cómo ni por qué

17. Da una opción, no cinco

❌

Puedes usar pytest, unittest, nose, tox o pytest-bdd según prefieras.

✅

Usa pytest. Si el proyecto ya usa otro corredor de pruebas, síguelo.

Ofrecer alternativas parece flexible y produce elecciones malas. Da una con salida de emergencia.

18. Escribe para el modelo más chico

Corre tu skill con Haiku, Sonnet y Opus. **Lo que basta para Opus puede quedarse corto para Haiku**. Una skill que depende de que el modelo "entienda la intención" se rompe en el modelo pequeño; una bien escrita funciona en los tres.

Es la prueba de robustez más barata que existe y casi nadie la hace.

19. Evalúa antes de documentar

La guía oficial invierte el orden habitual:

1. Corre la tarea **sin skill** y documenta dónde falla
2. Construye **tres escenarios** que prueben esas fallas
3. Mide la línea base
4. Escribe el **contenido mínimo** que las pasa

Es lo que evita skills que resuelven problemas imaginarios. Escribes para fallas observadas, no supuestas.

El tercer escenario es el que casi nadie hace: **el caso que NO debe disparar**. Una skill que se activa de más estorba tanto como una que no se activa.

20. Prueba siempre en sesión nueva

El contexto que quedó de cuando escribiste la skill **enmascara los huecos** de las instrucciones. Tú sabes lo que quisiste decir, y esa conversación también. Una sesión limpia no.

21. Diagnóstico rápido

COMANDO	QUÉ TE DICE
<code>/skills</code>	Qué hay cargado, de dónde, con qué metadatos
<code>/doctor</code>	Revisión de instalación y costo del listado
<code>/context</code>	Qué está ocupando la ventana
<code>claude --debug</code>	Errores de parseo del frontmatter
<code>claude plugin validate .claude/skills</code>	Encuentra frontmatter roto en todo un directorio

Prueba de descarte en tres pasos

Pregunta ¿qué skills tienes disponibles? :

- Aparece con descripción → el problema es de redacción
- Aparece sin descripción → el frontmatter no parsea
- No aparece → problema de ubicación o de visibilidad

Tabla de síntomas

SÍNTOMA	CAUSA MÁS PROBABLE
No se activa nunca	Descripción sin las palabras reales, o en 1ª/2ª persona
Responde a <code>/nombre</code> pero nunca sola	Frontmatter mal formado
Se activa cuando no debe	Descripción demasiado amplia
No encuentra un archivo	Ruta absoluta o barra invertida
Salta la solicitud de permiso	Falta declarar en <code>allowed-tools</code>
Funciona a veces sí y a veces no	Cadena de referencias de dos niveles
La invocación se aborta sin explicar	Un comando <code>!</code> falló. Pruébalo a mano
"Dejó de funcionar" a media conversación	Sigue en contexto; el modelo eligió otro camino

22. Antipatrones que no dan error

Los más costosos, porque nada te los señala:

ANTIPATRÓN	POR QUÉ FALLA
Ofrecer cinco librerías alternativas	Claude elige mal
"la versión actual es 3.2"	En seis meses tu skill miente con confianza
Cambiar de término a media skill	Claude no sabe si son lo mismo

ANTIPATRÓN	POR QUÉ FALLA
Explicar lo que Claude ya sabe	Costo recurrente sin beneficio
Referencias encadenadas	Funcionan a veces: peor que fallar siempre
<code>context: fork</code> en skill de solo lineamientos	Devuelve vacío en silencio
Narrar el porqué en vez de enunciar el qué	Ruido

23. Qué capa usar

NATURALEZA DEL CONTENIDO	CAPA
Hecho permanente del proyecto	<code>CLAUDE.md</code>
Procedimiento o material de referencia	Skill
Tarea con contexto aislado y rol propio	Subagente o <code>context: fork</code>
Regla que debe cumplirse siempre	Hook
Acceso a un sistema externo	Servidor MCP

Las dos preguntas que desempatan:

- **CLAUDE.md o skill** → ¿hace falta en *cada* conversación, o solo en *algunas*?
- **Skill o hook** → ¿confío en que el modelo *decida* aplicarlo? Si ya lo pusiste en CLAUDE.md y se sigue olvidando, es hook.

Documentación oficial

- [Mejores prácticas para la creación de Skills](#)
- [Extend Claude with skills](#)
- [Agent Skills overview](#)
- [agentskills.io](#) — estándar abierto

Claude Code cambia rápido. Verifica los campos del frontmatter y los comandos de diagnóstico contra estas fuentes antes de tomar decisiones importantes.